Software Systems

Lesson 11

Data and Control Structures in C

Slides – Vybihal & Bérubé-Vallières © 2023

Part 1

Variables

Readings: Textbook section 3.0 Example 2 and 4

Slides – Vybihal & Bérubé-Vallières © 2023

Basic Data Types (simplified)

- There are four basic types:
  - char
    - Represented as an integer. Usually 8 bits. Values depend on the machine's character set (On mimi you can expect 0 to 128 to correspond to ASCII)
  - int
    - Integer (whole number). C specifies that int must be at least 16 bits (-32,768 to 32,767), but on 32-bits and 64-bits machines it is usually 32 bits (-2,147,483,648 to 2,147,483,647).
  - float
    - Floating-point number (i.e. can have a fractional part). Usually 32 bits. Ranges about $1.18 \cdot 10^{-38}$ to $3.40 \cdot 10^{38}$ with 6 digits precision
  - double
    - Floating point number that is double the size of float. Ranges about $2.23 \cdot 10^{-308}$ to $1.80 \cdot 10^{308}$ with 15 digits precision
- void means no type
  - It is used as a function argument to say that it takes nothing, as a function return value to say that it returns nothing, and as a generic data pointer to an unknown type (later in course)

*(This slide simplifies the truth a bit, as all the combinations of modifier + type are in fact separate basic types. See slide 6)

Basic Data Types (simplified)

- There are four basic types:
  - char, int, float, double
- There are some modifiers that you can add in front of some of these types:
  - You can add signed or unsigned in front of char and int
    - Signed numbers can represent positive or negative numbers, while unsigned can only represent non-negative numbers
    - Without this modifier, int is signed, and char might be either
  - You can add short or long in front of int
    - A short int is useful when we want to conserve memory as it takes less space than an int to store (on 32-bits and 64-bits machines, typically 16 bits)
    - A long int is useful when we want numbers that are too large for int (on 64-bits machines, 64 bits)
- Integers are usually used for boolean testing
  - C99 introduced a _Bool type

*(This slide simplifies the truth a bit, as all the combinations of modifier + type are in fact separate basic types. See slide 6)

Basic Data Types (Full)

Type of number		Name	Other names	Min. bits	Range (minimum)
Integers	Unsigned	_Bool (C99)	bool (with stdbool.h)	-	0 to 1
		unsigned char		8	0 to 255
		unsigned short	unsigned short int	16	0 to 65535
		unsigned	unsigned int	16	0 to 65535
		unsigned long	unsigned long int	32	0 to 4,294,967,295
		unsigned long long (C99)	unsigned long long int	64	0 to 2^{64}-1
	Compiler-dependent	**char**		8	Compiler-dependent
	Signed	signed char		8	-128 to 127
		short	signed short short int signed short int	16	-32,768 to 32,767
		int	signed signed int	16	-32,768 to 32,767
		long	signed long long int signed long int	32	-2,147,483,648 to 2,147,483,647
		long long (C99)	signed long long long long int signed long long int	64	-2^{63} to 2^{63}-1
Floating-point	Real	**float**		- (32*)	$1.18 \cdot 10^{-38}$ to $3.40 \cdot 10^{38}$ (6 digits precision)
		double		- (64*)	$2.23 \cdot 10^{-308}$ to $1.80 \cdot 10^{308}$ (15 digits precision)
		long double		- (usually 80 or 128 bits)	
	Complex	float _Complex (C99)	float complex (with complex.h)	Equivalent to an array type containing two elements of the corresponding real type	
		double _Complex (C99)	double complex (with complex.h)		
		long double _Complex (C99)	long double_complex (with complex.h)		

*Not technically defined in C, but compilers usually follow IEEE 754 Standard for Floating-Point Arithmetic

Actual sizes depend on implementation

- Compilers are only required to follow minimum sizes. Implementations usually depend on CPU
  - On a 16-bit machine, short is typically 16 bits, int is 16 bits, and long is 32 bits
  - On a 32-bit machine, short is typically 16 bits, int is 32 bits, and long is 32 bits
  - On a 64-bit machine, short is typically 16 bits, int is 32 bits, and long is 64 bits (mimi is 64 bits)
- You can see the size of a type by doing sizeof()
  - For example, sizeof(int), sizeof(char), sizeof(float)

Slides – Vybihal & Bérubé-Vallières © 2023

Derived data types

- All other data types in C are derived from the previous basic types
- There are four ways to derive data types, which we will see later in this course:
  - Arrays: Combine items with the same type
  - Structures: Combine items with different types
  - Pointer: Address to an object in memory
  - Others not covered in this course.

- Strings do not have their own type. They are either char * (an address to a character) or char[] (an array of characters)

Slides – Vybihal & Bérubé-Vallières © 2023

Variable Declaration (Simple)

A variable declaration that doesn't use storage classes or qualifiers (later in course) or derived data types has the following syntax:

Syntax:

TYPE VAR_NAME;

TYPE VAR_NAME = VALUE;

TYPE VAR1, VAR2, … , VARn;

Where:

TYPE - char, int, float, double, void, etc.

VAR_NAME - must start with a character, followed by digits, characters, and symbols (not reserved)

Later in the course you will see how certain specifiers can be added in front of the type like extern, etc., how pointers are used in front of variable names, or how a variable name can be followed by [n] to specify the size of an array

Examples

- int x;
- int x, y, z;
- int x = 5, y, z = 2;
- short int a = -2;
- unsigned short int b = 4;
- char c = 4, d ='x'; // char c=4; char d='x';

Constant Declarations

- In C, a variable can be declared as constant.
- The value of a constant is initialized when the variable is declared. That value cannot be changed.
- Convention is generally to have these variables be uppercase

```
int const A = 1;
const int A = 2;
```


Slides – Vybihal & Bérubé-Vallières © 2023

typedef Declaration

The typedef command allows for the creation of custom type names. This makes the program more readable.

```
typedef int scalefactor;   // a simple example

int main() {

    scalefactor a;

    a = 10;

    printf("The scale factor is:%d", a);
}
```


Slides – Vybihal & Bérubé-Vallières © 2023

typedef Declaration

```
typedef int boolean;
Const int true=1, false=0;
boolean isValid = false;

typedef double gravity;
gravity mars, jupiter;

typedef float money;
money bankAccount;
```

Operators

Math

Assuming integer math

•	+	add	x = 5 + 2;	// x becomes 7
•	-	subtract	x = 5 – 2;	// x becomes 3
•	*	multiply	x = 5 * 2;	// x becomes 10
•	/	divide	x = 5 / 2;	// x becomes 2
•	%	modulo	x = 5 % 2;	// x becomes 1
•	++	increment	x++;	// if x=5 then x=6
•	--	decrement	x--;	// if x=5 then x=4
•	+=	increment by	x += 3;	// if x=5 then x=8
•	-=	decrement by	x -= 3;	// if x=5 then x=2
•	*=	multiply by	x *= 3;	// if x=5 then x=15
•	/=	divide by	x /= 3;	// if x=5 then x=1

Operators

Logical

• <	less than	5 < 10	true
• >	greater than	5 > 10	false
• <=	less and equal	5 <= 5	true
• >=	greater and equal	5 >= 5	true
• ==	equal	5 == 10	false
• !=	not equal	5 != 10	true
• !	Not	! (5 == 10)	true
• &&	and	(5<10)&&(5>10)	false
• ||	or	(5<10)||(5>10)	true

Expressions

With assignment

- VARIABLE = EXPRESSION;
  - x = 5 + y; // x will contain 5 more than y
  - x = 5 > 10; // results in 1 for true, or 0 for false

Without assignment

- (EXPRESSION)
  - if (x < 10) // true when x is less than 10
  - if (x + 2) // true when result is not equal to zero

Notice the low-level features of C, where logical expressions and integer mathematics mix.

Slides – Vybihal & Bérubé-Vallières © 2023

Expressions

What do the following output?

- Incrementing and Decrementing
  - x = x + y++; // assume x = 5 and y = 2
  - x = x + ++y;

Standard C definition:
- Var++ → increment after solving the expression
- ++Var → increment before solving the expression

- Conditional Assignment Statement
  - VAR = (CONDITION) ? TRUE_EXPRESSION : FALSE_EXPRESSION;
  - z = (y < 10) ? x++ : x--; // assume x = 5 and y = 2

Expressions

What do the following output?

- Since Boolean is integer in C…

```
if (5) printf("Does this print?");

if (0) printf("Does this print?");

if (5+2-25) printf("Does this print?")

int x=5, y=7;
if (x + y) printf("Does this print?");

#include<math.h>
if (sqrt(25)) printf("Does this print?");
```


Slides – Vybihal & Bérubé-Vallières © 2023

Expressions

What do the following output?

- Characters are integer too..

```
char x = 'A';
printf("%c", x);

x = x + 1;
printf("%c", x);
```


Slides – Vybihal & Bérubé-Vallières © 2023

Part 2

C Control Structures

Slides – Vybihal & Bérubé-Vallières © 2023

The if-statement

if (CONDITION) SINGLE_STATEMENT;

```
if (x < 10) puts("X is less than 10\n");
```

if (CONDITION) { MULTIPLE_STATEMENTS; }

```
if (x < 10) {
    puts("X is less than 10\n");
    c = getc(stdin);
}
```

The if-statement

if (CONDITION) SINGLE_STATEMENT; else SINGLE_STATEMENT;

```
    if (x < 10) puts("X is less than 10\n");
    else puts("X is greater than 10\n");
```

if (CONDITION) { MULTIPLE_STATEMENTS; }

else {MULTIPLE_STATEMENTS;}

```
    if (x < 10) {
        puts("X is less than 10\n");
        c = getc(stdin);
    } else {
        puts("X is greater than 10\n");
    }
```


Slides – Vybihal & Bérubé-Vallières © 2023

The switch-statement

```
switch(VARIABLE) {
    case VALUE:     MULTIPLE_STATEMENTS;
                    break;
    case VALUE2:    MULTIPLE_STATEMENTS;
                    break;
    default:
            MULTIPLE_STATEMENTS;
}
```

Optional. It designates the end of the case block. If not present execution automatically goes to the next case block without testing the condition.

The VARIABLE can only be of type integer or character.

The VALUE is a constant, it cannot be a variable.

The switch-statement

```
switch(age) {
 case 1:  puts("You are too young,sorry!\n");
      break;
 case 2:
 case 3: puts("Bring a parent.\n");
      break;
 default:
    puts("You can drive the car.\n");
}
```

We are assuming that the variable AGE is an integer.
If the value is a 1 then the first case is activated only.
If the value is a 2 or 3 then the second case is activated.
All other integer values are handled by the default case.

The switch-statement

```
switch(student) {
 case 'y':
 case 'Y':
        puts("Welcome\n");
        break;
 case 'n':
 case 'N':
        puts("Only students are welcome\n");
        break;
 default:
    puts("Please enter an Y or an N.\n");
}
```

The while-loop

while (CONDITION_IS_TRUE) SINGLE_STATEMENT;

while (CONDITION_IS_TRUE) { MULTIPLE_STATEMENTS; }

```
int x = 0;
while (x < 10) x++;
```

```
int x = 10;
while(x--);
```

```
int y = 20;
while (y > 0) {
puts("Hi! ");
y--;
}
```

The do-while-loop

do SINGLE_STATEMENT; while (CONDITION_IS_TRUE);

do { MULTIPLE_STATEMENTS; } while (CONDITION_IS_TRUE);

```
char specie;
do {
     puts("Specie (Elf or Hobbit): ");
     specie = getc(stdin);
} while (specie!='E' && specie!='e');
```

The for-loop

```
for (START; CONDITION; EXPRESSION) SINGLE_STATEMENT;
```

```
for (START; CONDITION; EXPRESSION) {
    MULTIPLE_STATEMENTS;
}
```

The START is run before the loop starts,
the CONDITION is checked before each iteration and the loop runs as long as it evaluates to true
the EXPRESSION is run after each iteration

START and EXPRESSION can be comma-separated lists.

START, CONDITION, and EXPRESSION are optional !!

```
for(;;);

 for(;x<10;) x--;
```


Slides – Vybihal & Bérubé-Vallières © 2023

What does this output?

```
int x;
for(x=0; x<10; x++) puts("Hi!");
```

```
int x, y;
char c;
for(x=0, y=10, c=' '; x<10 && c != 'x'; x+=2, y--) {
 puts("Hi");
 c = getc(stdin);
}
```